

IdeaDNA

AI-Powered Idea Evolution Platform

"Don't just generate ideas. Evolve them."

Document Type:	Product Requirements Document (PRD)
Version:	2.0 — Resume-Grade Production Build
Date:	June 2026
Author:	Platform Developer
Status:	Final — Ready for Development

Platform Statement

IdeaDNA is an AI-powered Idea Evolution Platform that helps users transform problems into innovative solutions, evolve existing ideas into multiple optimized versions, reverse-engineer developer thinking from code, and generate complete project blueprints tailored to their goals, skills, and constraints. The platform implements a lazy-loading architecture where heavy assets (blueprints, UI mockups) are generated only after user selection, optimizing cost and performance.

■ Table of Contents

- 1. Problem Statement
- 2. Vision & Mission
- 3. Target Users & Personas
- 4. Core Features & Architecture
- 5. User Flow & Interaction Design
- 6. Database Schema & Data Model
- 7. API Design & Backend Architecture
- 8. UI/UX Design System
- 9. AI Agent Architecture (Multi-Agent System)
- 10. Performance & Optimization Strategy
- 11. Security & Authentication
- 12. Future Roadmap
- 13. Success Metrics & KPIs
- 14. Tech Stack & Dependencies
- 15. Implementation Timeline

1. Problem Statement

Students, startup founders, and developers face critical challenges in the innovation pipeline:

- **Ideation Gap:** Struggling to find innovative, non-obvious project ideas that stand out from competitors
- **Evolution Stagnation:** Existing AI tools generate ideas but fail to evolve them into multiple optimized versions
- **Thinking Opacity:** No way to understand how experienced developers think and architect solutions from existing code
- **Blueprint Vacuum:** Generated ideas lack actionable implementation plans, tech stacks, and database schemas
- **Resource Waste:** Generating heavy assets (code, mockups) for all ideas when users only select one
- **Personalization Deficit:** Generic outputs that ignore user's skill level, budget, team size, and timeline

Current Market Gap:

Most AI tools are 'one-shot generators' — they produce a list of ideas and stop. None provide a structured evolution pipeline, reverse-engineering capability, or on-demand deep-dive generation. IdeaDNA fills this gap with a multi-agent, lazy-loading, visualization-first architecture.

2. Vision & Mission

Vision

To build the world's first AI-powered innovation platform that doesn't just generate ideas — it evolves them, archives them, and teaches users how to think like world-class developers through reverse-engineering and structured evolution.

Mission

Empower every developer, founder, and student to go from 'I have a problem' to 'I have a production-ready blueprint' in under 10 minutes, while learning the thinking patterns behind great software architecture.

3. Target Users & Personas

■■■ Hackathon Harry

College student, 20 years old. Needs a unique project idea in 24 hours. Wants something that impresses judges. Skill level: Intermediate. Budget: \$0. Team: 3 people.

■ Startup Sarah

Aspiring founder, 26 years old. Has identified a problem but needs to validate multiple solution approaches. Wants to see evolved versions and pick the most scalable one. Skill level: Advanced. Budget: \$5000. Team: 2 people.

■ Learner Leo

Self-taught developer, 22 years old. Wants to understand how experienced developers think by analyzing real projects. Reverse-engineers GitHub repos to learn architecture decisions. Skill level: Beginner. Budget: \$0. Team: Solo.

■ Product Manager Priya

PM at a tech company, 30 years old. Needs to quickly generate PRDs and tech stack recommendations for stakeholder meetings. Skill level: Advanced. Budget: Company-funded. Team: 5-10 people.

4. Core Features & Architecture

■ Mode 1: Idea Generator

User inputs a problem or theme → AI generates 5-8 ranked idea cards with innovation scores, feasibility ratings, and difficulty levels. Each card is lightweight (text only). User selects ONE card to unlock deep-dive generation.

■ Mode 2: Evolution Engine

User inputs an existing idea → AI generates 4-6 evolved versions (AI-Enhanced, Blockchain-Enhanced, Analytics-Enhanced, Enterprise-Enhanced, etc.). Visualized as an interactive DNA evolution tree. User selects ONE version for deep-dive.

■ Mode 3: Code Archaeology

User inputs code snippet, GitHub URL, or live website → AI reverse-engineers the developer's thinking process, reconstructs the idea evolution path, identifies architecture decisions and trade-offs, and generates an innovation score with future evolution suggestions.

■ Deep Dive: Blueprint Generator

Triggered ONLY after user selection. Generates: Project Overview, PRD, Tech Stack, Database Schema, API Design, User Flow, Implementation Plan. Exportable as Markdown or PDF.

■ Deep Dive: UI Mockup Generator

Triggered ONLY after user selection. Generates HTML/CSS mockup with responsive previews (Desktop, Mobile, Tablet). Exportable as standalone HTML file.

■ Innovation Scoring Engine

Multi-dimensional scoring: Originality (0-100), Technical Feasibility (0-100), Market Potential (0-100), Complexity Match (based on user profile). Visualized as interactive radar chart.

■ User Profiles & Personalization

Skill level, budget, team size, timeline, and technology preferences. AI adapts outputs dynamically. Beginner → Simple MVP ideas. Advanced → Research-level solutions.

■ Project Persistence

Users can save projects, revisit evolution history, and build a personal idea library. Vector search for finding similar past ideas.

5. User Flow & Interaction Design

Primary User Journey (Idea Generator Mode)

- **Step 1 — Mode Selection:** User lands on dashboard and selects one of three modes: ■ Generate, ■ Evolve, or ■ Analyze
- **Step 2 — Input:** User describes a problem (e.g., 'Students struggle to find internships') or pastes code/URL
- **Step 3 — Lightweight Generation:** AI generates 5-8 idea cards with scores. Each card contains: Title, Description, Problem Statement, Solution, Key Features, Innovation Score, Feasibility Score, Difficulty Level
- **Step 4 — Selection:** User clicks ONE card. This triggers the deep-dive pipeline. Unselected cards remain accessible but do not generate heavy assets
- **Step 5 — Deep Dive Unlock:** Selected idea expands to show two tabs: ■ Blueprint and ■ UI Mockup
- **Step 6 — Blueprint View:** Structured sections: Overview, PRD, Tech Stack, Database Schema, API Design, User Flow, Implementation Timeline. Exportable
- **Step 7 — UI Mockup View:** Live HTML/CSS preview with device toggle (Desktop/Mobile/Tablet). Exportable as HTML file
- **Step 8 — Action:** User can: ■ Evolve this idea further, ■ Save to project library, ■ Export blueprint, or ■ Start new session

Lazy-Loading Architecture Rationale

Heavy assets (blueprints, UI mockups) are generated on-demand after user selection. This reduces API costs by ~80%, improves response time for initial generation, and ensures only relevant content is produced. Each unselected idea remains a lightweight card (text + scores) consuming minimal storage.

6. Database Schema & Data Model

Prisma Schema — PostgreSQL

```
// Core Entities model User { id String @id @default(uuid()) email String @unique
name String? skillLevel String // BEGINNER | INTERMEDIATE | ADVANCED budget Int? //
in USD teamSize Int? timeline String? // e.g., "2 weeks", "3 months" techPrefs
String[] // ["React", "Node", "Python"] projects Project[] createdAt DateTime
@default(now()) } model Project { id String @id @default(uuid()) userId String user
User @relation(fields: [userId], references: [id]) mode Mode // GENERATOR |
EVOLUTION | ARCHAEOLOGY status Status // DRAFT | SELECTED | COMPLETED originalInput
String // Problem text / Idea text / Code input ideas Idea[] selectedIdeaId String?
@unique selectedIdea Idea? @relation("SelectedIdea", fields: [selectedIdeaId],
references: [id]) blueprint Blueprint? uiMockup UiMockup? evolutionTree Json? //
For EVOLUTION mode archaeologyReport Json? // For ARCHAEOLOGY mode createdAt
DateTime @default(now()) updatedAt DateTime @updatedAt } model Idea { id String @id
@default(uuid()) projectId String project Project @relation(fields: [projectId],
references: [id]) title String description String problemStatement String solution
String keyFeatures String[] innovationScore Int feasibilityScore Int difficulty
String // EASY | MEDIUM | HARD tags String[] selectedBy Project?
@relation("SelectedIdea") blueprint Blueprint? uiMockup UiMockup? createdAt
DateTime @default(now()) } model Blueprint { id String @id @default(uuid()) ideaId
String @unique idea Idea @relation(fields: [ideaId], references: [id]) projectId
String @unique project Project @relation(fields: [projectId], references: [id])
overview String prd Json // Structured PRD sections techStack Json // Frontend,
Backend, Database, AI, DevOps databaseSchema Json // Tables, relations, fields
apiDesign Json // Endpoints, methods, payloads, auth userFlow Json // Step-by-step
user journey implementationPlan Json // Phases, milestones, timeline markdown
String // Exportable raw markdown createdAt DateTime @default(now()) } model
UiMockup { id String @id @default(uuid()) ideaId String @unique idea Idea
@relation(fields: [ideaId], references: [id]) projectId String @unique project
Project @relation(fields: [projectId], references: [id]) html String // Full HTML
mockup css String // Stylesheet components Json // Breakdown: Header, Hero,
Features, Footer mobileNotes String? // Responsive considerations createdAt
DateTime @default(now()) } enum Mode { GENERATOR, EVOLUTION, ARCHAEOLOGY } enum
Status { DRAFT, SELECTED, COMPLETED }
```


7. API Design & Backend Architecture

REST API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	User registration	Public
POST	/api/auth/login	User login	Public
GET	/api/projects	List all user projects	Required
POST	/api/projects	Create new project (mode selection)	Required
GET	/api/projects/:id	Get project with ideas	Required
POST	/api/projects/:id/generate	Generate ideas (GENERATOR mode)	Required
POST	/api/projects/:id/evolve	Generate evolved versions	Required
POST	/api/projects/:id/analyze	Analyze code (ARCHAEOLOGY)	Required
POST	/api/ideas/:id/select	Select idea → trigger deep-dive	Required
GET	/api/ideas/:id/blueprint	Get blueprint (lazy-loaded)	Required
GET	/api/ideas/:id/mockup	Get UI mockup (lazy-loaded)	Required
POST	/api/ideas/:id/evolve-further	Evolve selected idea more	Required
GET	/api/export/blueprint/:id	Export blueprint as PDF/Markdown	Required
GET	/api/export/mockup/:id	Export UI mockup as HTML	Required

AI Pipeline Architecture

All AI operations are handled via LangChain AgentExecutor with SequentialChain. Each mode has a dedicated pipeline:

- **GENERATOR Pipeline:** ProblemParser → IdeaGeneratorAgent → ScorerAgent → OutputFormatter
- **EVOLUTION Pipeline:** IdeaParser → CriticAgent → EvolverAgent (×4 variants) → ScorerAgent → TreeBuilder
- **ARCHAEOLOGY Pipeline:** CodeParser → PatternDetector → IdeaHistorian → DevPsychologist → ScorerAgent → EvolverAgent
- **BLUEPRINT Pipeline:** SelectedIdea → BlueprintArchitect → TechStackRecommender → SchemaDesigner → APIDesigner → TimelinePlanner
- **MOCKUP Pipeline:** SelectedIdea + Blueprint → UIDesigner → HTMLGenerator → CSSGenerator → ResponsiveAdapter

8. UI/UX Design System

Design Philosophy

Dark-first theme with 'DNA Helix' visual metaphor. Clean, futuristic, developer-friendly. Minimal clutter, maximum information density.

Color Palette

Primary: #1a1a2e (Deep Navy), Secondary: #16213e (Midnight), Accent: #e94560 (Crimson), Success: #00d9ff (Cyan), Background: #0f0f23 (Void Black)

Typography

Headings: Inter Bold, Body: Inter Regular, Code: JetBrains Mono. Scale: 28px (Hero) → 18px (H1) → 14px (H2) → 12px (Body) → 10px (Caption)

Component Library

Custom components: DNA Helix Loader, Evolution Tree (React Flow), Radar Chart (D3.js), Idea Cards with hover effects, Blueprint Accordion, Device Toggle (Desktop/Mobile/Tablet)

Animations

Framer Motion: Page transitions, card hover lifts, tree node expansions, score counter animations, DNA strand pulse effects

Responsive Breakpoints

Mobile: 320px-767px (stacked cards, simplified tree), Tablet: 768px-1023px (2-column grid), Desktop: 1024px+ (full experience with side panels)

9. AI Agent Architecture (Multi-Agent System)

The platform uses a Multi-Agent orchestration system built on LangChain. Each agent has a specialized role, system prompt, and output schema.

■ Generator Agent

Role: Transforms problem statements into solution concepts. System prompt optimized for creative divergence. Temperature: 0.8. Output: Array of idea objects with title, description, features.

■ Critic Agent

Role: Analyzes ideas for weaknesses, gaps, and risks. System prompt optimized for critical analysis. Temperature: 0.3. Output: Weakness report with severity ratings.

■ Evolver Agent

Role: Takes Critic's feedback and generates upgraded versions. System prompt optimized for structured mutation. Temperature: 0.6. Output: Evolved idea variants with mutation tags.

■ Scorer Agent

Role: Evaluates ideas on multi-dimensional rubric. System prompt optimized for objective scoring. Temperature: 0.2. Output: JSON with originality, feasibility, market_potential, complexity_match scores.

■ Blueprint Architect

Role: Generates comprehensive project documentation. System prompt optimized for technical writing. Temperature: 0.4. Output: Structured PRD with all sections.

■ UI Designer Agent

Role: Generates HTML/CSS mockups from blueprint. System prompt optimized for modern UI patterns. Temperature: 0.5. Output: Complete HTML file with inline CSS.

■ Pattern Detector

Role: Identifies design patterns and architecture from code. System prompt optimized for code analysis. Temperature: 0.3. Output: Pattern list with confidence scores.

■ Idea Historian

Role: Reconstructs idea evolution timeline from code commits/structure. System prompt optimized for historical reasoning. Temperature: 0.5. Output: Evolution timeline with version descriptions.

■■■■ Dev Psychologist

Role: Infers developer thinking process and trade-offs. System prompt optimized for cognitive reasoning. Temperature: 0.4. Output: Thinking process report with decision rationale.

10. Performance & Optimization Strategy

Lazy Loading (Core Strategy)

Heavy assets (blueprints, UI mockups) generated only after user selection. Initial idea cards are text-only (~500 bytes each). Reduces API costs by ~80% and improves TTFB (Time to First Byte) by ~60%.

Streaming Responses

AI outputs streamed via Server-Sent Events (SSE) for real-time user experience. Users see text appearing character-by-character instead of waiting for full response.

Caching Layer

Redis cache for: frequently generated ideas, popular problem themes, blueprint templates. Cache TTL: 24 hours for ideas, 1 hour for scores.

Vector Search

Pinecone/Supabase Vector for semantic similarity search across user's past ideas. Enables 'Similar to your idea X...' recommendations.

Database Indexing

Indexes on: userId (project queries), projectId (idea queries), mode (filtering), status (dashboard filtering). Composite index on (userId, createdAt) for project history.

Image/Asset Optimization

UI mockups served as static HTML files (no SSR needed). CDN caching for exported assets. Lazy loading for blueprint images/diagrams.

Rate Limiting

Tiered limits: Free tier: 10 generations/day, Pro tier: 100 generations/day, Enterprise: Unlimited. Prevents API abuse and controls costs.

11. Security & Authentication

Authentication

Clerk/NextAuth.js with OAuth 2.0 (Google, GitHub). JWT tokens with 7-day expiry. Refresh token rotation.

Authorization

RBAC (Role-Based Access Control): User (own projects only), Admin (all projects). Middleware-level route protection.

Data Protection

PII encrypted at rest (AES-256). API keys stored in environment variables, never exposed to client. OpenAI API calls server-side only.

Input Sanitization

All user inputs sanitized via Zod validation. Code inputs scanned for malicious patterns before processing. SQL injection prevention via Prisma ORM parameterized queries.

Rate Limiting

API rate limits per user: 10 requests/minute for free, 60/minute for pro. IP-based rate limiting for public endpoints.

12. Future Roadmap

Phase 2 (Month 2-3)

Team Matching System, Real-time Collaboration, Idea Marketplace, Chrome Extension for quick idea capture

Phase 3 (Month 4-6)

Investor Readiness Score, Startup Validation Module, Market Research Generator, Competitive Analysis Agent, Pitch Deck Auto-Builder

Phase 4 (Month 7-12)

AI Co-Founder Mode, Multi-language Support, Enterprise API, White-label Solution, Mobile App (React Native)

13. Success Metrics & KPIs

Metric	Target	Measurement
Ideas Generated	10,000+/month	Database count
Evolution Rate	40%+	Evolved ideas / Total ideas
Selection Rate	25%+	Selected ideas / Generated ideas
Blueprint Exports	15%+	Exports / Selected ideas
User Retention (7-day)	30%+	Return users / Total users
Avg. Session Duration	8+ minutes	Analytics
User Satisfaction	4.5/5	Post-session survey
API Cost per Idea	<\$0.05	OpenAI usage tracking

14. Tech Stack & Dependencies

Category	Technology	Version	Purpose
Frontend	Next.js	14.x	App Router, SSR, API Routes
Frontend	TypeScript	5.x	Type safety
Frontend	Tailwind CSS	3.x	Styling
Frontend	Framer Motion	11.x	Animations
Frontend	React Flow	12.x	Evolution tree graphs
Frontend	D3.js	7.x	Radar charts, data viz
Frontend	Zustand	4.x	State management
Frontend	React Query	5.x	Server state
Backend	Next.js API Routes	14.x	REST API
Backend	Prisma ORM	5.x	Database operations
Backend	PostgreSQL	15.x	Primary database
Backend	Redis	7.x	Caching, sessions
AI	LangChain	0.1.x	Agent orchestration
AI	OpenAI GPT-4	API	LLM inference
AI	Pinecone	Latest	Vector search
Auth	Clerk	Latest	Authentication
Deploy	Vercel	—	Frontend hosting
Deploy	Railway/Render	—	Database + Redis

15. Implementation Timeline

Week	Focus	Deliverables
Week 1	Project Setup & Auth	Next.js scaffold, Clerk auth, Prisma schema, DB setup, basic layout
Week 2	Mode 1: Idea Generator	Problem input, AI pipeline, idea cards, scoring, selection flow
Week 3	Mode 2: Evolution Engine	Evolution tree (React Flow), variant generation, visual DNA graph
Week 4	Mode 3: Code Archaeology	Code parser, GitHub integration, archaeology report UI, reverse-engineering
Week 5	Deep Dive: Blueprint	Blueprint generator, PRD sections, tech stack, database schema, API design
Week 6	Deep Dive: UI Mockup	HTML/CSS generator, device preview, responsive toggle, export
Week 7	Polish & Optimization	Animations, caching, streaming, performance tuning, bug fixes
Week 8	Testing & Deploy	E2E tests, load testing, Vercel deploy, documentation, portfolio prep

— End of Document —